K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Name of Student : Shlok Mitesh Rawal
Roll No : B - 73
Batch : BS4

EXPERIMENT 6

Aim:

To Implement Binary Tree and its Traversal for real-world application

Objective :

- To understand the concept and structure of a Binary Tree in Data Structures and Algorithms (DSA).
- To learn how nodes are created and connected to form a binary tree. ● To implement different binary tree traversal techniques, namely Inorder, Preorder, and Postorder traversal.
- To understand the practical applications of binary trees in real-world scenarios, such as hierarchical data organization and decision-making systems. ● To develop a program that demonstrates the construction and traversal of a binary tree.

Theory :

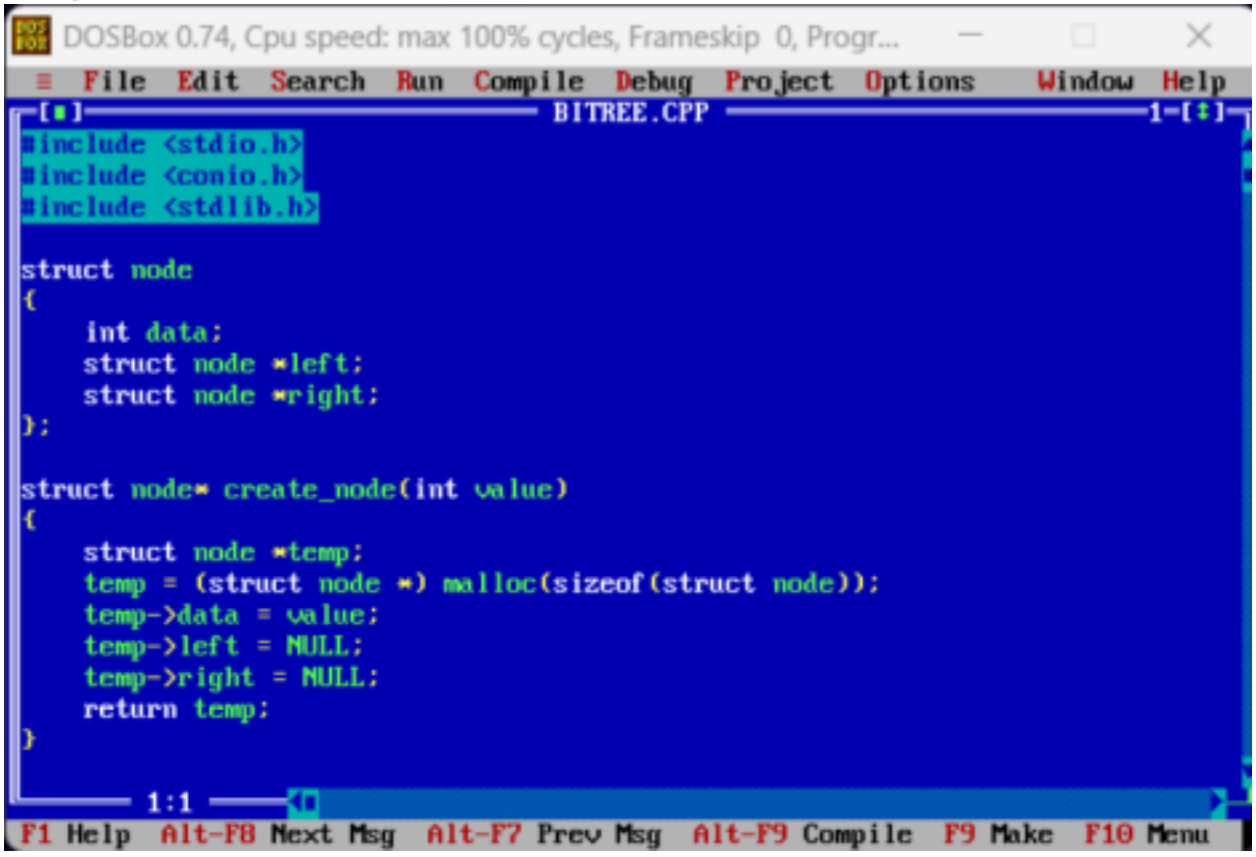
A **Binary Tre**e is a non-linear data structure in which each node can have at most two children, known as the left child and the right child. The topmost node is called the root, while nodes that do not have any children are called leaf nodes. Binary trees are widely used to represent hierarchical relationships and organize data efficiently.

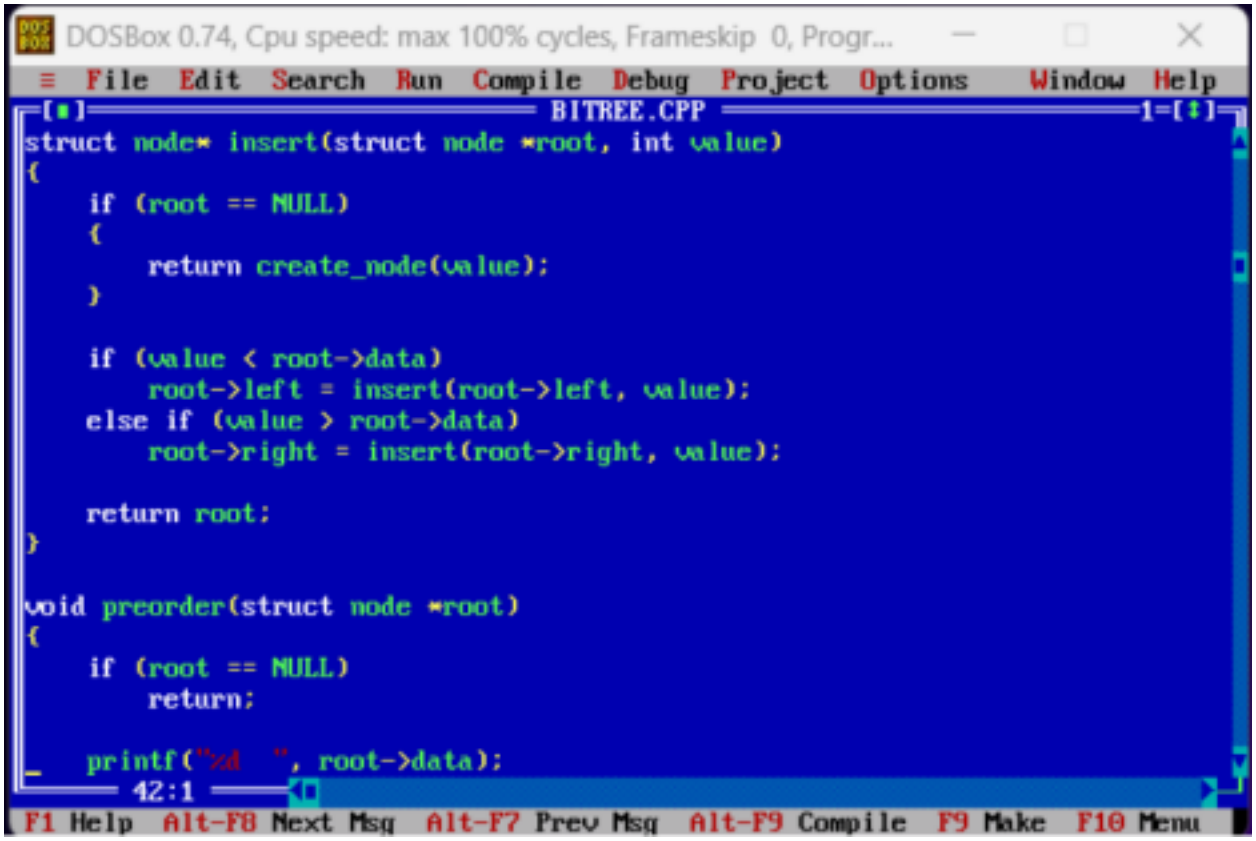
Tree traversal is the process of visiting every node in a binary tree in a specific order. The three common depth-first traversal techniques are **Inorder, Preorder, and Postorder traversal.**

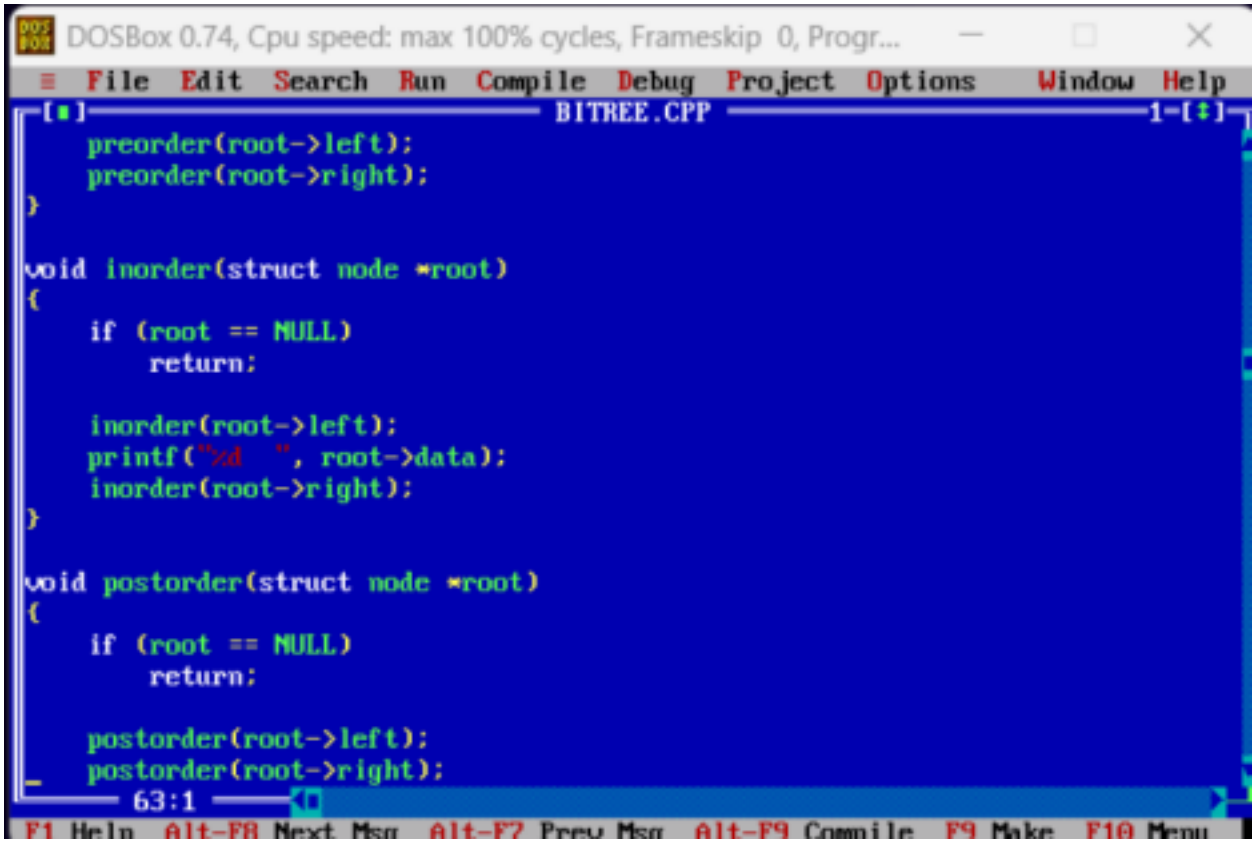
In **Inorder Traversal**, nodes are visited in the order Left, Root, Right. In Preorder Traversal, nodes are visited in the order Root, Left, Right. In Postorder Traversal, nodes are visited in the order Left, Right, Root. These techniques are useful for processing tree data in different sequences.

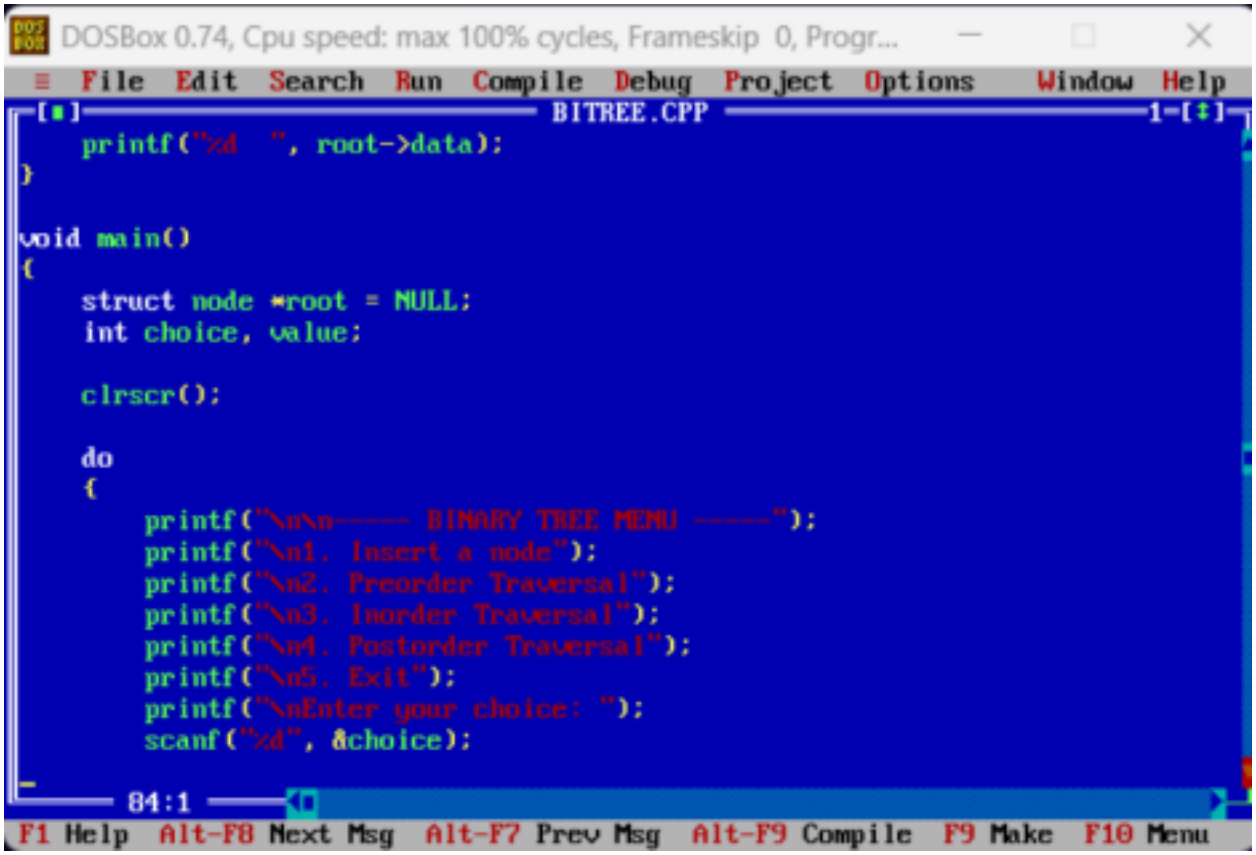
Binary trees have several real-world applications. For example, they can represent decision-making processes, file-system hierarchies, and expression evaluation in mathematical calculations. In a binary search tree, data can also be organized to support efficient searching, insertion, and deletion.

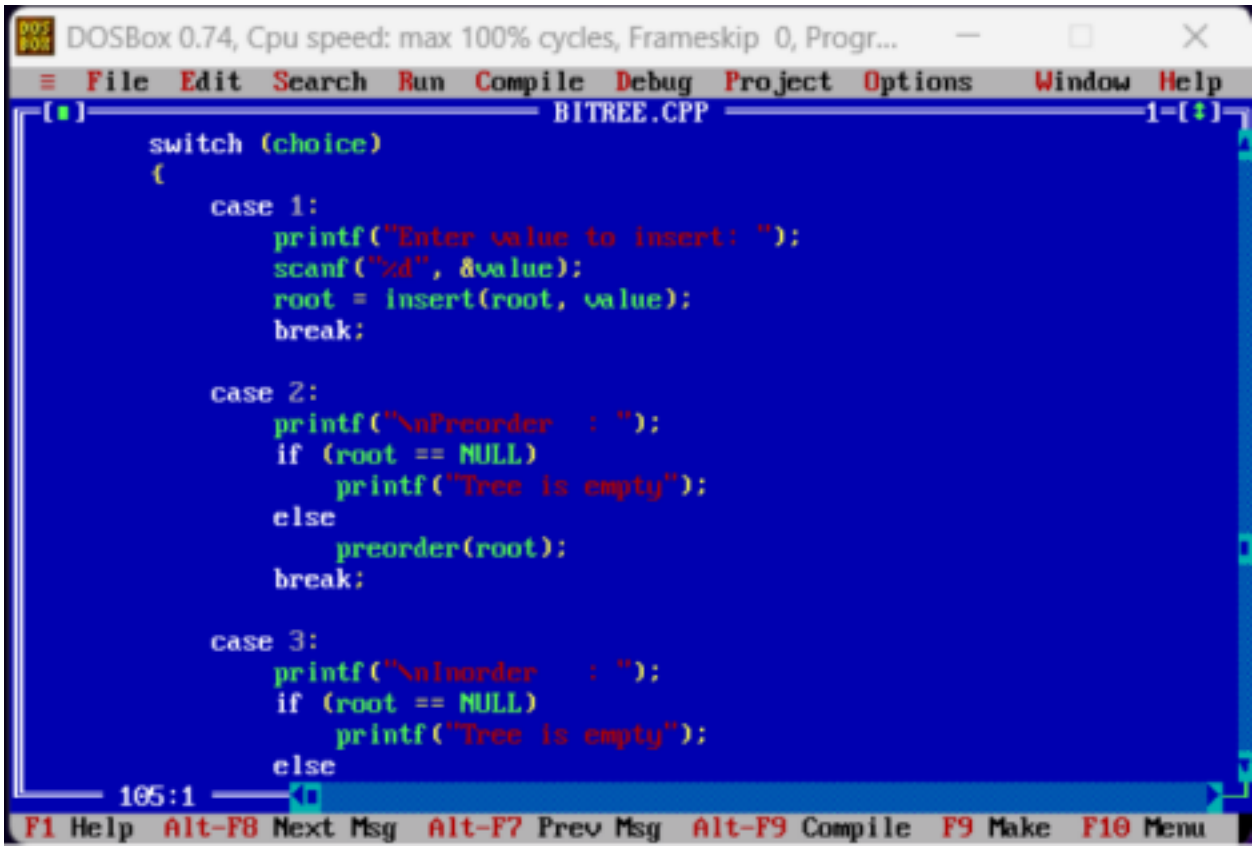
Program :

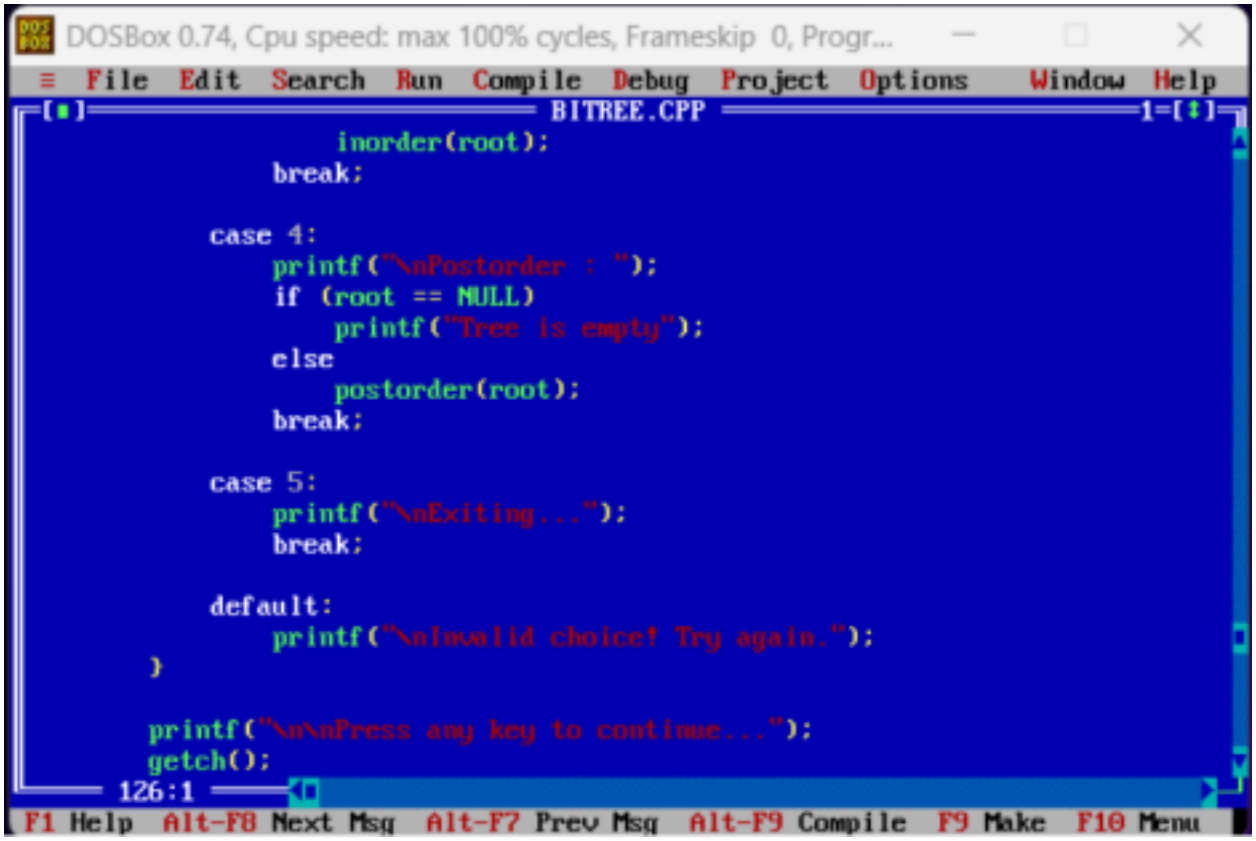


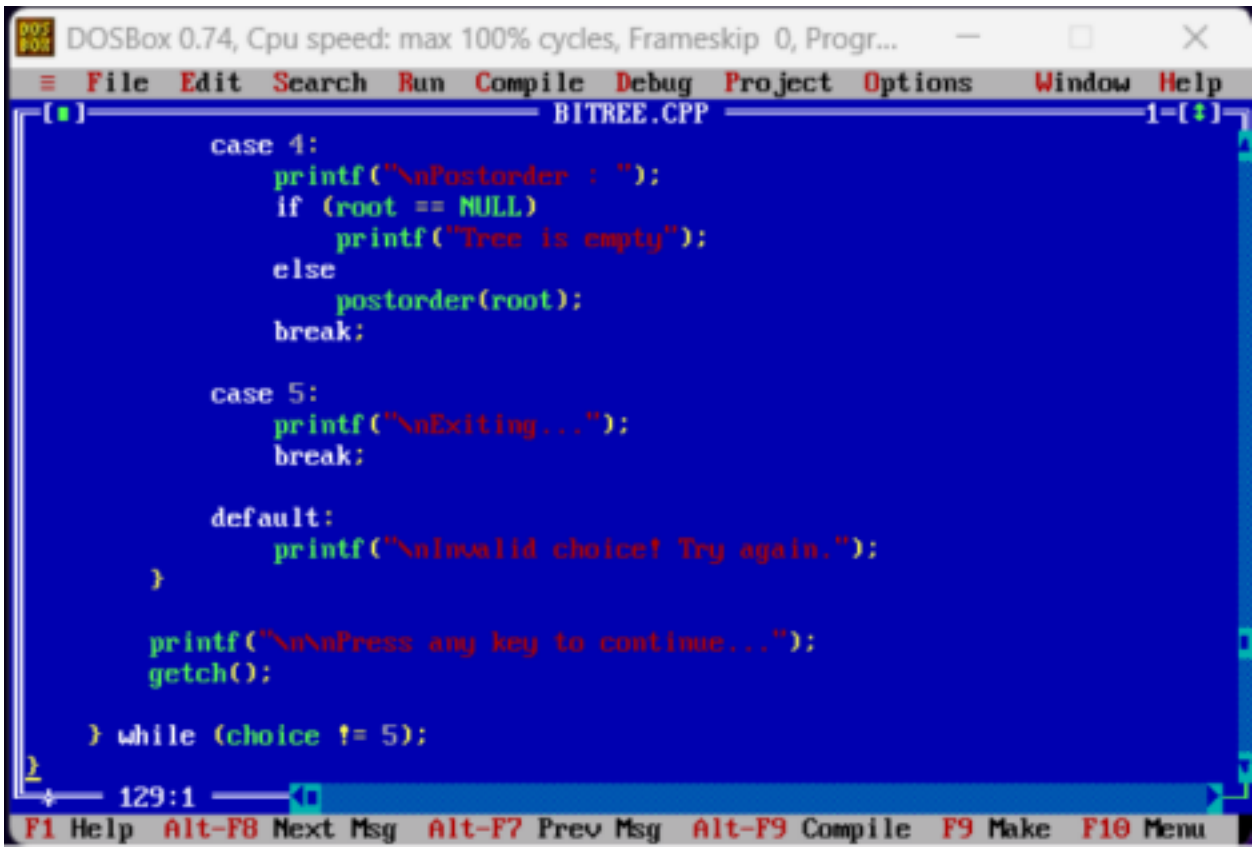

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
File Edit Search Run Compile Debug Project Options Window Help
BITREE.CPP
1=[↕]
struct node* insert(struct node *root, int value)
{
if (root == NULL)
{
return create_node(value);
}
if (value < root->data)
root->left = insert(root->left, value);
else if (value > root->data)
root->right = insert(root->right, value);
return root;
}
void preorder(struct node *root)
{
if (root == NULL)
return;
printf("%d ", root->data);
42:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu


DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
File Edit Search Run Compile Debug Project Options Window Help
BITREE.CPP
1=[↕]
preorder(root->left);
preorder(root->right);
}
void inorder(struct node *root)
{
if (root == NULL)
return;
inorder(root->left);
printf("%d ", root->data);
inorder(root->right);
}
void postorder(struct node *root)
{
if (root == NULL)
return;
postorder(root->left);
postorder(root->right);
63:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

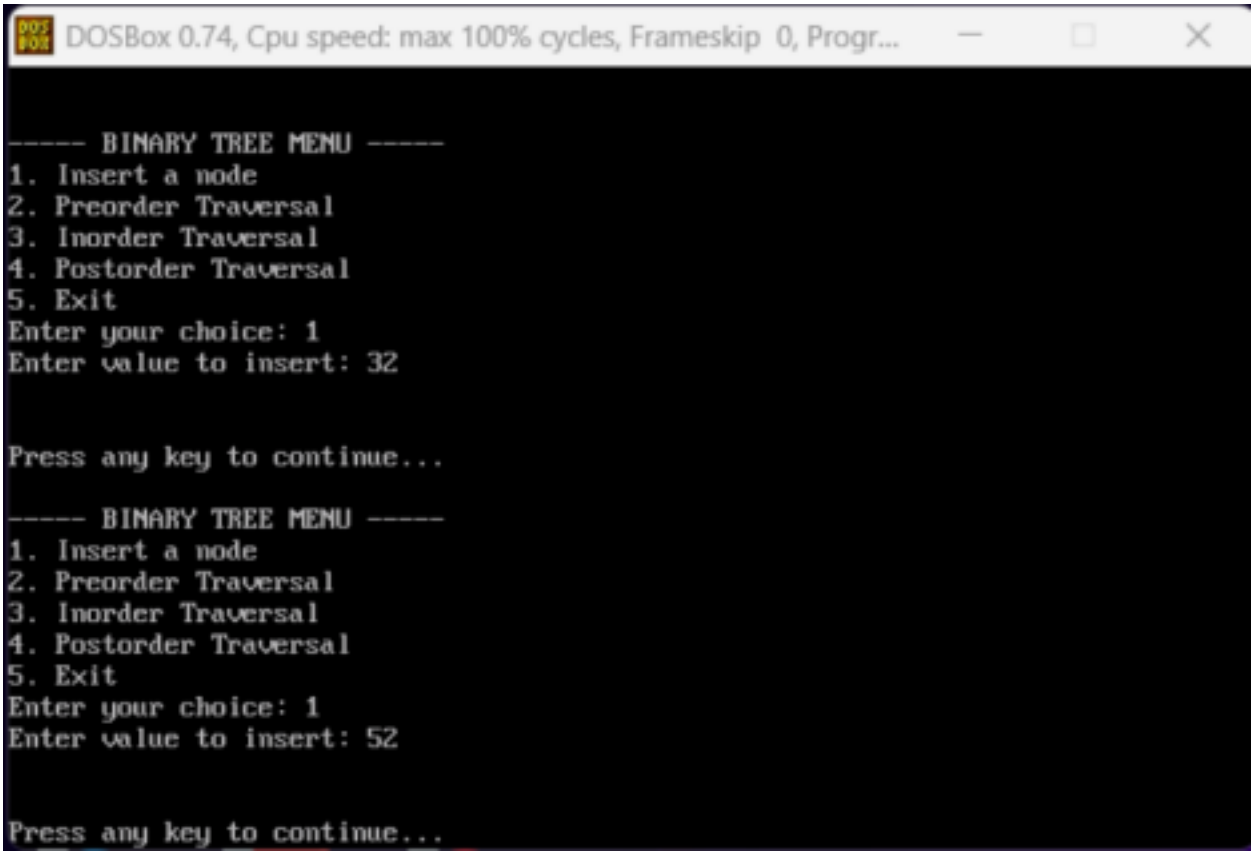

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
File Edit Search Run Compile Debug Project Options Window Help
BITREE.CPP
printf("%d ", root->data);
}
void main()
{
struct node *root = NULL;
int choice, value;
clrscr();
do
{
printf("\n\n----- BINARY TREE MENU -----");
printf("\n1. Insert a node");
printf("\n2. Preorder Traversal");
printf("\n3. Inorder Traversal");
printf("\n4. Postorder Traversal");
printf("\n5. Exit");
printf("\nEnter your choice: ");
scanf("%d", &choice);
84:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

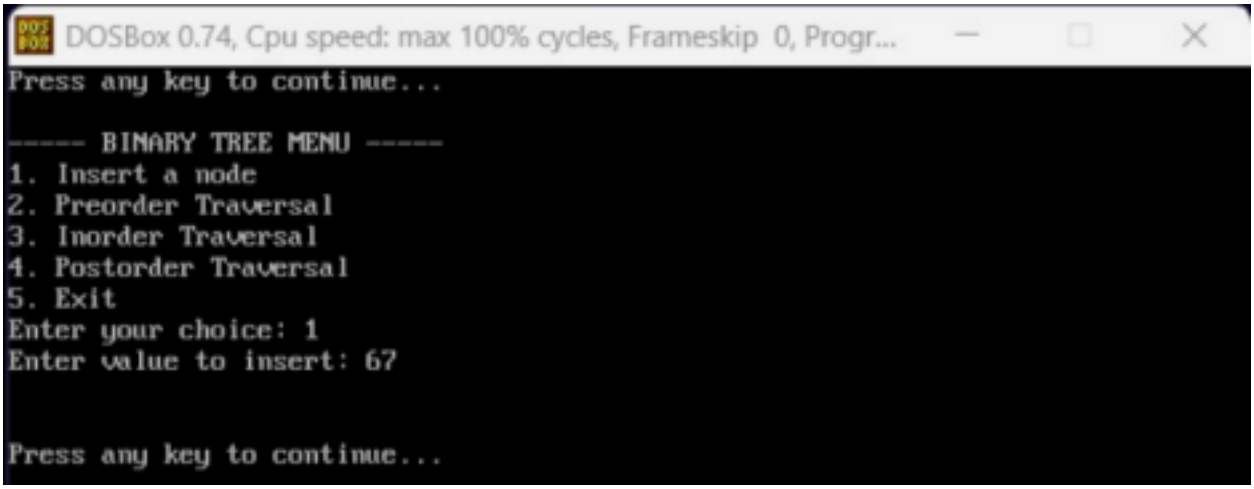

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
File Edit Search Run Compile Debug Project Options Window Help
BITREE.CPP
switch (choice)
{
case 1:
printf("Enter value to insert: ");
scanf("%d", &value);
root = insert(root, value);
break;
case 2:
printf("\nPreorder : ");
if (root == NULL)
printf("Tree is empty");
else
preorder(root);
break;
case 3:
printf("\nInorder : ");
if (root == NULL)
printf("Tree is empty");
else
105:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

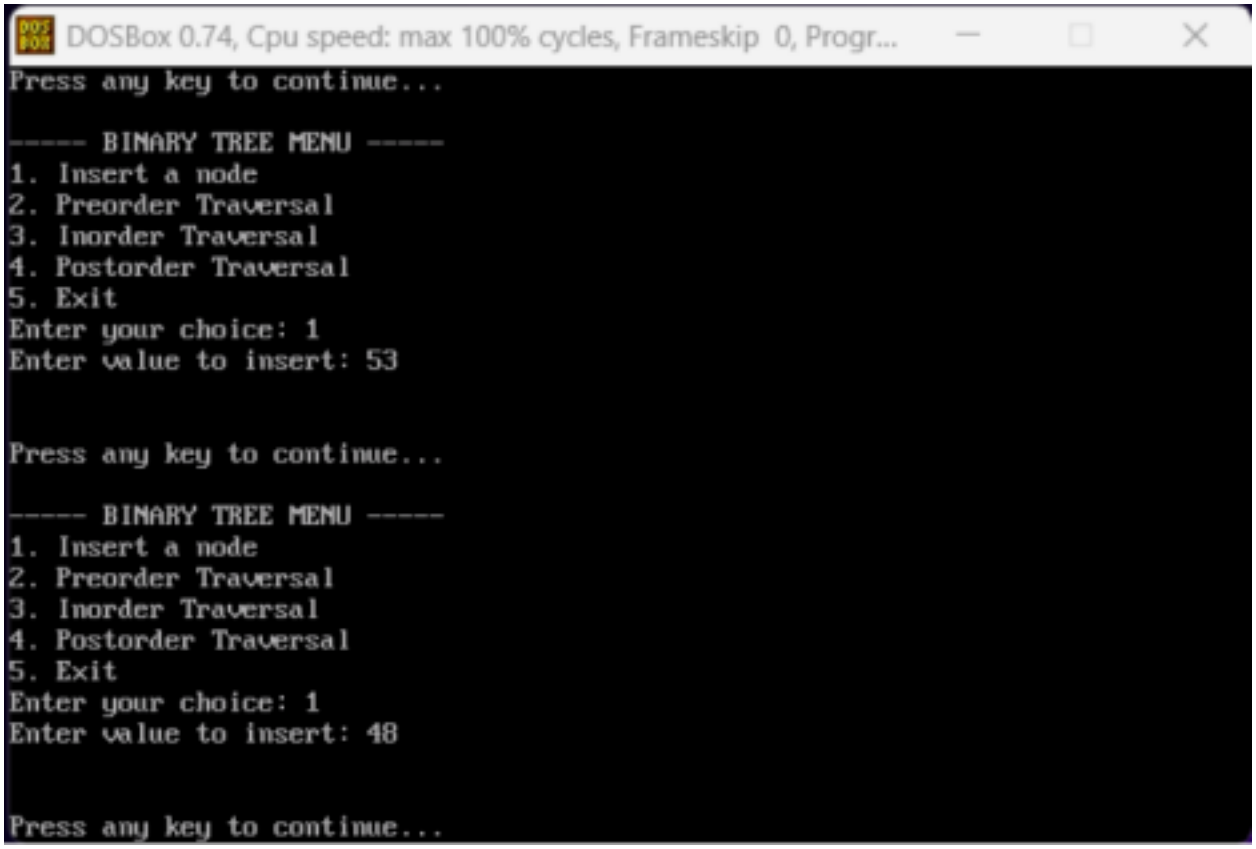

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
File Edit Search Run Compile Debug Project Options Window Help
BITREE.CPP
inorder(root);
break;
case 4:
printf("\nPostorder : ");
if (root == NULL)
printf("Tree is empty");
else
postorder(root);
break;
case 5:
printf("\nExiting...");
break;
default:
printf("\nInvalid choice! Try again.");
}
printf("\n\nPress any key to continue...");
getch();
126:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

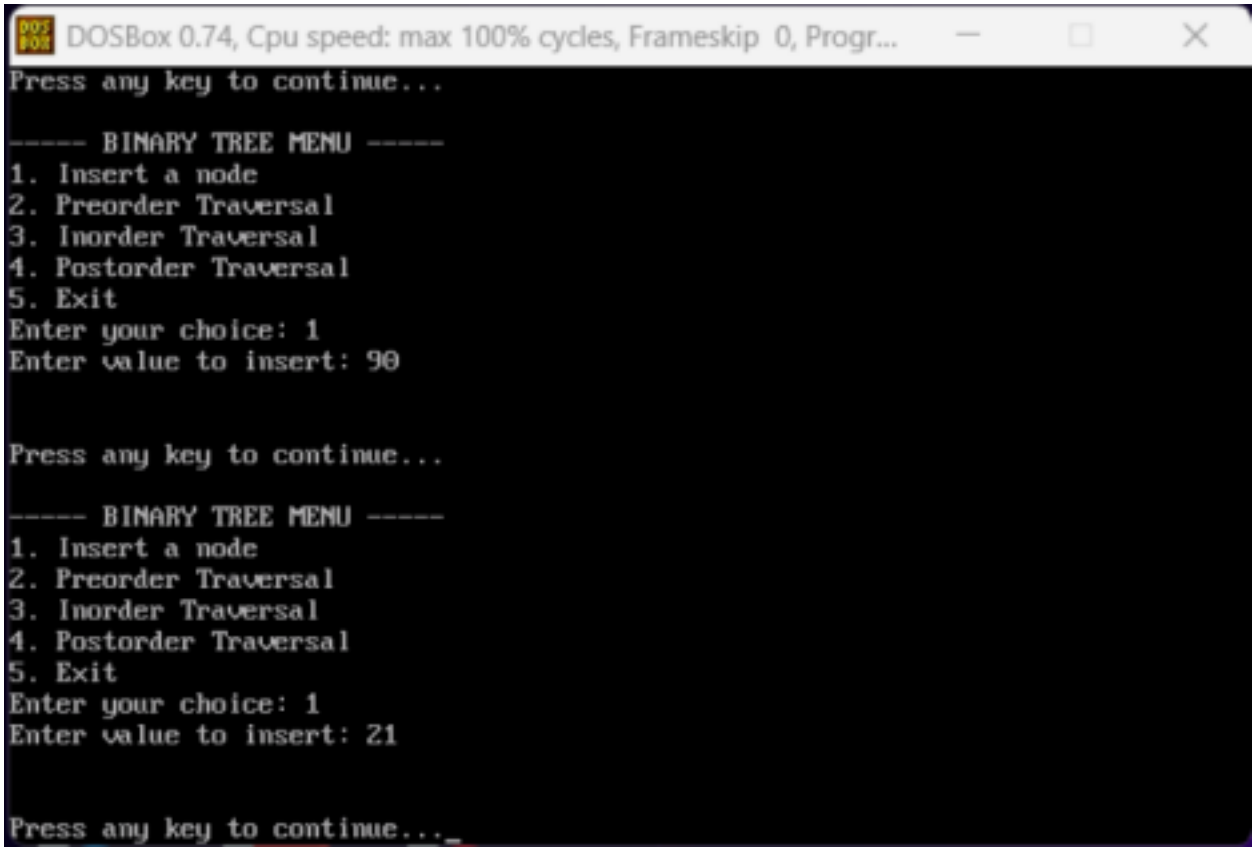

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
File Edit Search Run Compile Debug Project Options Window Help
BITREE.CPP
case 4:
printf("\nPostorder : ");
if (root == NULL)
printf("Tree is empty");
else
postorder(root);
break;
case 5:
printf("\nExiting...");
break;
default:
printf("\nInvalid choice! Try again.");
}
printf("\n\nPress any key to continue...");
getch();
} while (choice != 5);
}
129:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

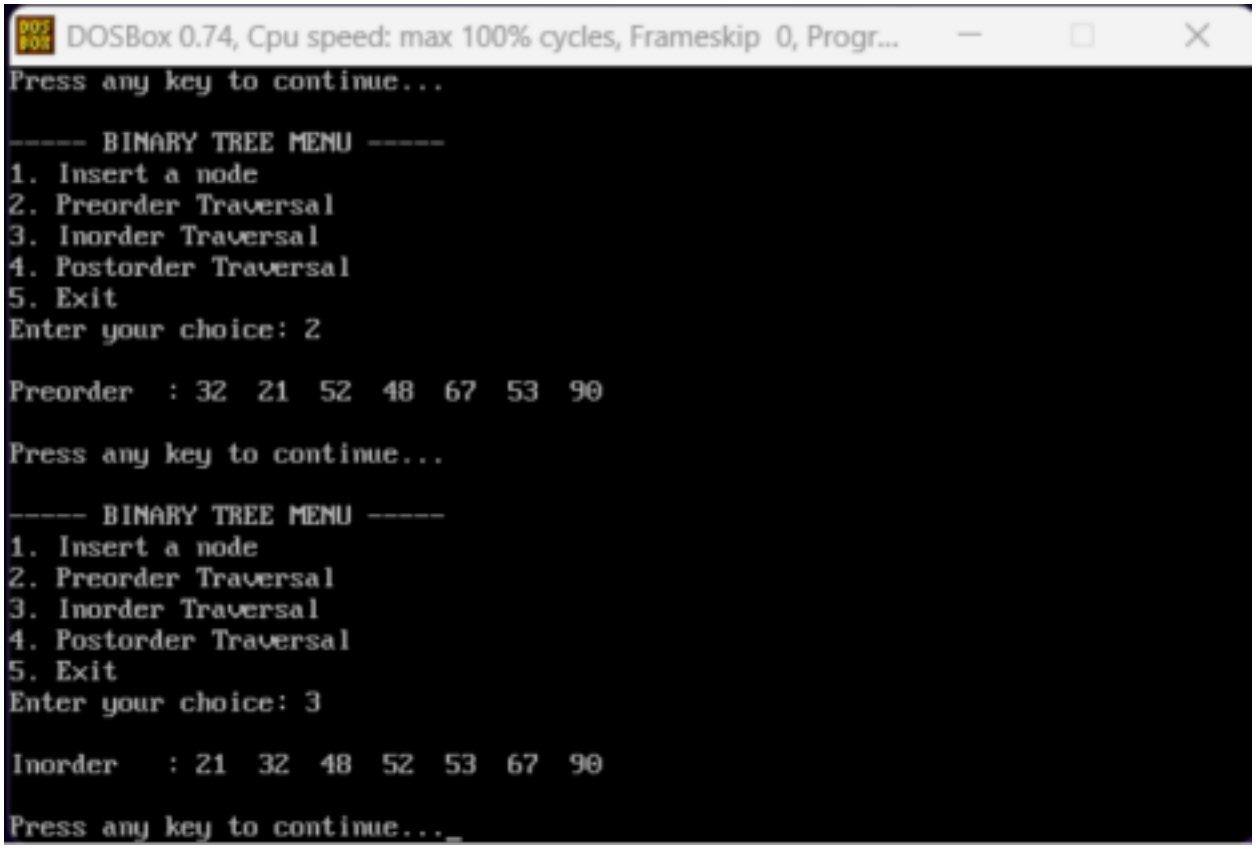
Output :

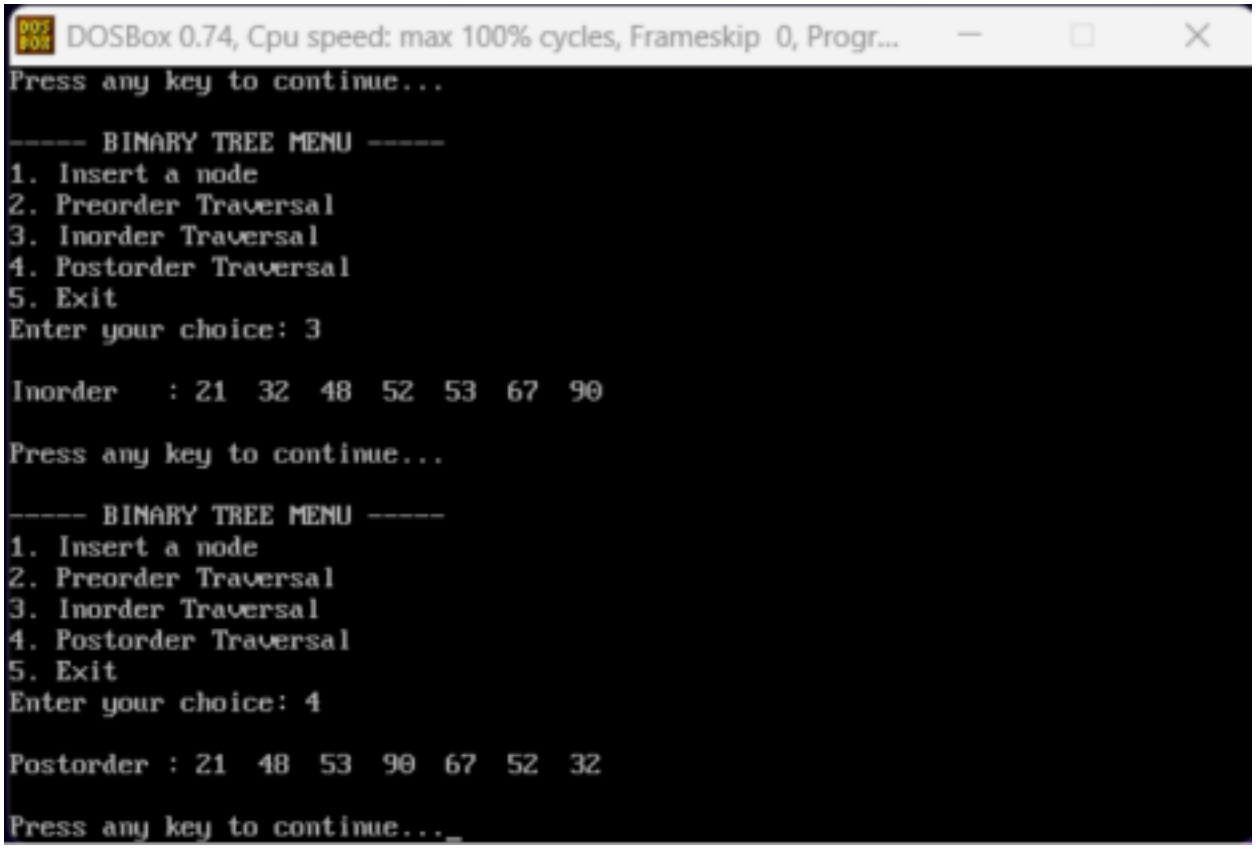

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 32
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 52
Press any key to continue...

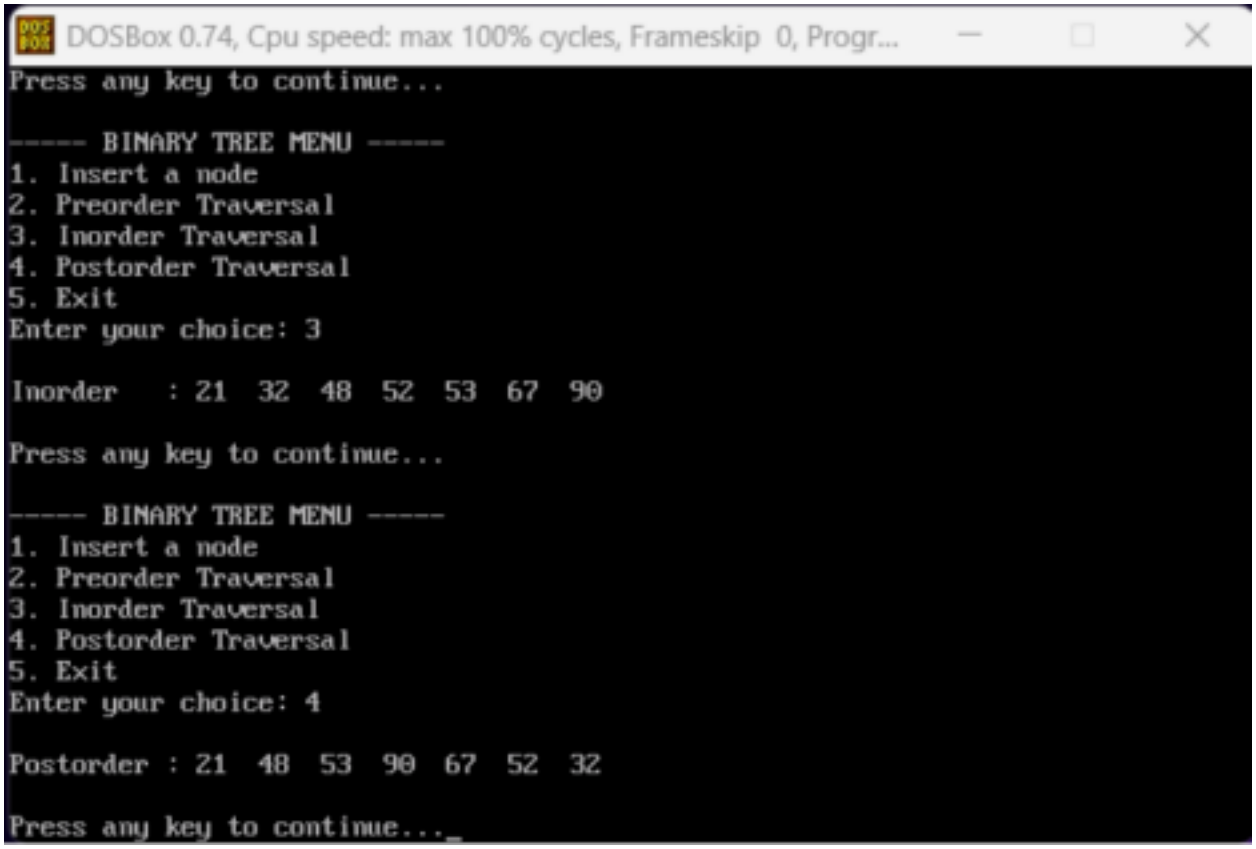

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 67
Press any key to continue...


DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 53
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 48
Press any key to continue...


DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 90
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 21
Press any key to continue...


DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 2
Preorder : 32 21 52 48 67 53 90
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 3
Inorder : 21 32 48 52 53 67 90
Press any key to continue...


DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 3
Inorder : 21 32 48 52 53 67 90
Press any key to continue...
----- BINARY TREE MENU -----
1. Insert a node
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 4
Postorder : 21 48 53 90 67 52 32
Press any key to continue...



Outcome :

The binary tree data structure was studied and implemented successfully. The creation of tree nodes and the relationships between parent and child nodes were understood. The Inorder, Preorder, and Postorder traversal techniques were implemented to visit

and process the nodes in different sequences. The practical also demonstrated the importance of binary trees in organizing hierarchical data and solving real-world problems.

Conclusion :

Thus, the implementation of a Binary Tree and its Traversal Techniques was studied successfully. The practical provided an understanding of binary tree construction and the Inorder, Preorder, and Postorder traversal methods. It also highlighted the importance of binary trees in hierarchical data representation, decision-making, and efficient data processing in real-world applications.